МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №3
по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU

Выполнил: Ефимов Сергей Юрьевич
Группа: М80-403Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы. Научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.

Формат изображений соответствует формату описанному в лабораторной работе 2. Во всех вариантах, в результирующем изображении, на месте альфа-канала должен быть записан номер класса(кластера) к которому был отнесен соответствующий пиксель. Если пиксель можно отнести к нескольким классам, то выбирается класс с наименьшим номером.

Входные данные.

На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению.

На следующей строке, число nc -- количество классов. Далее идут nc строчек описывающих каждый класс. В начале j-ой строки задается число npj -- количество пикселей в выборке, за ним следуют npj пар чисел -- координаты пикселей выборки. $nc <= 32$, $np_j <= 2^{19}$, $w * h <= 4 * 10^8$

Оценка вектора средних и ковариационной матрицы:

$$avg_j = \frac{1}{np_j - 1} \sum_{i=1}^{np_j} ps_i^j$$

$$cov_j = \frac{1}{np_j - 1} \sum_{i=1}^{np_j} (ps_i^j - avg_j) * (-avg_j)^T ps_i^j$$

Где $ps_i^j = \left(r_i^j, g_i^j, b_i^j\right)^T$ -- i-ый пиксель из j-ой выборки.

Вариант 3. Метод минимального расстояния.

Для некоторого пикселя p, номер класса jc определяется следующим образом:

$$jc = \arg \max(-(p - avg_j)^T * (p - avg_j))$$

Программное и аппаратное обеспечение

Видеокарта

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 615.71.09              KMD Version: 615.71.09     CUDA UMD Version: 13.4
|
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================
+======================|
|   0  NVIDIA GeForce GTX 1660 ...    Off |   00000000:01:00.0  On |                  N/A |
| 43%   31C    P8             11W /  125W |     557MiB /   6144MiB |     16%      Default |
|                                         |                        |                  N/A |
```

```
+-----------------------------------------+----------------------+----------------------+
Device 0
Compute capability      : 7.5
Name                    : NVIDIA GeForce GTX 1660 SUPER
Total Global Memory     : 6034096128
Shared memory per block : 49152
Registers per block     : 65536
Warp size               : 32
Max threads per block   : (1024, 1024, 64)
Max block               : (2147483647, 65535, 65535)
Total constant memory   : 65536
Multiprocessors count   : 22

Процессор
Архитектура:             x86_64
  CPU op-mode(s):        32-bit, 64-bit
  Address sizes:         39 bits physical, 48 bits virtual
  Порядок байт:          Little Endian
CPU(s):                  12
  On-line CPU(s) list:   0-11
ID прроизводителя:       GenuineIntel
  Имя модели:            11th Gen Intel(R) Core(TM) i5-11500 @ 2.70GHz
    Семейство ЦПУ:       6
    Модель:              167
    Thread(s) per core:  2
    Ядер на сокет:       6
    Сокетов:             1
    Степпинг:            1
    Microcode version:   0x66
    CPU(s) scaling MHz:  92%
    CPU max MHz:         4600,0000
    CPU min MHz:         800,0000
    BogoMIPS:            5424,00

Оперативная память
               total        used        free      shared  buff/cache   available
Mem:            15Gi       7,0Gi       1,4Gi       237Mi       7,8Gi       8,5Gi
Swap:           15Gi       6,2Mi        15Gi

Жесткий диск
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    0 931,5G  0 disk
└─sda1        8:1    0   931G  0 part
sr0          11:0    1  1024M  0 rom
```

```
zram0       253:0    0  15,4G  0 disk [SWAP]
nvme0n1     259:0    0   1,9T  0 disk
├─nvme0n1p1 259:1    0     4G  0 part /boot
└─nvme0n1p2 259:2    0   1,9T  0 part /var/tmp
                                /var/log
                                /var/cache
                                /home
                                /root
                                /srv
                                /
```

Система
Linux cachyos-x8664 7.2.6-1-cachyos #1 SMP PREEMPT_DYNAMIC Thu, 17 Sep 2026 15:20:14 +0000 x86_64 GNU/Linux

IDE
CLion от JetBrains

Компилятор:
Для локального тестирования использовался CMake со следующей конфигурацией:

```
cmake_minimum_required(VERSION 4.3)
set(CMAKE_CUDA_COMPILER /opt/cuda/bin/nvcc)
set(CUDAToolkit_ROOT /opt/cuda)
project(pgp_tpp LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 14)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

При отправке использовался makefile

```
NVCC = /usr/bin/nvcc
CFLAGS = --std=c++14
SOURCES = lab3.cu
BIN = lab3
all:
	$(NVCC) $(CFLAGS) -o $(BIN) $(SOURCES)
```

Метод решения

Для реализации классификации изображения по методу минимального расстояния с использованием CUDA нужна функция с __global__ для запуска обработки изображения на GPU. В функции происходит распределение потоков по пикселям. Каждый поток вычисляет квадраты евклидовых расстояний от цвета пикселя до средних цветов классов в пространстве RGB и выбирает класс с минимальным расстоянием. Номер вы-

бранного класса записывается в четвёртый байт пикселя, а его цвет сохраняется. В главной функции main происходит считывание изображения и координат обучающих пикселей каждого класса, по которым на CPU вычисляются средние цвета классов. Средние цвета копируются в константную память GPU, затем выделяется память для изображения и выполняется перенос пикселей из CPU в GPU. После запуска функции классификации результат переносится из GPU в CPU и сохраняется в выходной файл. В завершение программа освобождает выделенную память на GPU.

Описание программы

Функция classifyKernel необходима для классификации пикселей изображения по методу минимального расстояния с использованием CUDA. Внутри используются blockIdx (номер блока), blockDim (количество потоков внутри блока) и threadIdx (номер потока) для определения линейного индекса пикселя. Каждый поток обрабатывает пиксели в цикле с шагом, равным общему количеству запущенных потоков. Для каждого пикселя вычисляются квадраты евклидовых расстояний между его цветом и средними цветами классов, хранящимися в константной памяти classAverages. Выбирается класс с минимальным расстоянием; при равенстве расстояний предпочтение отдаётся классу с меньшим номером. Номер класса записывается в четвёртый байт пикселя, а компоненты RGB сохраняются. В главной функции main считываются имена входного и выходного файлов, затем функция readImage загружает размеры и пиксели изображения. Далее считываются количество классов и координаты обучающих пикселей, по которым вычисляется средний цвет каждого класса. С помощью cudaMemcpyToSymbol средние цвета переносятся в константную память GPU. Функция cudaMalloc выделяет память для изображения, а cudaMemcpy копирует в неё пиксели из CPU. Задаются блоки по 256 потоков и рассчитывается количество блоков, ограниченное числом 65535. После запуска classifyKernel проверяются ошибки с помощью cudaGetLastError, а cudaDeviceSynchronize ожидает завершения обработки. Затем cudaMemcpy переносит результат из GPU в CPU, cudaFree освобождает память GPU, а функция writeImage сохраняет изображение в выходной файл.

Результаты

Время работы ядер с различными конфигурациями

Image: 256 x 256, classes: 8

Blocks	Threads	Time (ms)	Result
1	32	3.850605	OK
1	64	2.521578	OK
1	128	2.184701	OK
1	256	2.207520	OK
1	512	2.095415	OK
1	1024	2.094566	OK
2	32	1.768912	OK
2	64	1.245504	OK
2	128	0.678483	OK

2	256	0.582058	OK
2	512	0.582269	OK
2	1024	0.582730	OK
4	32	0.484826	OK
4	64	0.343206	OK
4	128	0.296358	OK
4	256	0.291805	OK
4	512	0.292042	OK
4	1024	0.292330	OK
8	32	0.243990	OK
8	64	0.173104	OK
8	128	0.149827	OK
8	256	0.147478	OK
8	512	0.146614	OK
8	1024	0.146326	OK
16	32	0.122493	OK
16	64	0.088090	OK
16	128	0.075869	OK
16	256	0.074579	OK
16	512	0.074640	OK
16	1024	0.074355	OK
32	32	0.087421	OK
32	64	0.077530	OK
32	128	0.074726	OK
32	256	0.074506	OK
32	512	0.076253	OK
32	1024	0.075008	OK
64	32	0.062042	OK
64	64	0.057322	OK
64	128	0.057210	OK
64	256	0.056701	OK
64	512	0.057069	OK
64	1024	0.057008	OK
128	32	0.056992	OK
128	64	0.058170	OK
128	128	0.056218	OK
128	256	0.056202	OK
128	512	0.056822	OK
128	1024	0.057021	OK
256	32	0.056445	OK
256	64	0.056730	OK
256	128	0.056509	OK
256	256	0.056413	OK
256	512	0.056410	OK
256	1024	0.057715	OK
512	32	0.056394	OK
512	64	0.056464	OK
512	128	0.056563	OK

512	256	0.056784	OK
512	512	0.056397	OK
512	1024	0.058672	OK
1024	32	0.056646	OK
1024	64	0.056954	OK
1024	128	0.056720	OK
1024	256	0.056698	OK
1024	**512**	**0.056358**	**OK**
1024	1024	0.063216	OK

Image: 1024 x 1024, classes: 16

Blocks	Threads	Time (ms)	Result
1	32	59.758217	OK
1	64	42.086723	OK
1	128	35.668167	OK
1	256	35.812080	OK
1	512	36.261726	OK
1	1024	36.345364	OK
2	32	30.378637	OK
2	64	21.031914	OK
2	128	18.243599	OK
2	256	18.190769	OK
2	512	18.042103	OK
2	1024	17.757214	OK
4	32	14.833255	OK
4	64	10.594784	OK
4	128	9.327101	OK
4	256	9.088449	OK
4	512	9.011057	OK
4	1024	8.885565	OK
8	32	7.417306	OK
8	64	5.381133	OK
8	128	4.653389	OK
8	256	4.621613	OK
8	512	4.776080	OK
8	1024	4.576093	OK
16	32	3.867827	OK
16	64	2.766106	OK
16	128	2.326903	OK
16	256	2.297584	OK
16	512	2.271561	OK
16	1024	2.241091	OK
32	32	2.674752	OK
32	64	2.309702	OK
32	128	2.242278	OK
32	256	2.241974	OK
32	512	2.279242	OK
32	1024	2.259402	OK

64	32	1.734624	OK
64	64	1.693863	OK
64	128	1.684240	OK
64	256	1.683111	OK
64	512	1.683008	OK
64	1024	1.683181	OK
128	32	1.693680	OK
128	64	1.728192	OK
128	128	1.682557	OK
128	256	1.682166	OK
128	512	1.681734	OK
128	1024	1.683622	OK
256	32	1.683158	OK
256	64	1.682096	OK
256	128	1.682208	OK
256	256	1.681456	OK
256	512	1.682189	OK
256	1024	1.684099	OK
512	32	1.681463	OK
512	64	1.682112	OK
512	128	1.680352	OK
512	256	1.679789	OK
512	512	1.679158	OK
512	1024	1.685865	OK
1024	32	1.679245	OK
1024	64	1.679757	OK
1024	128	1.645350	OK
1024	256	1.644989	OK
1024	**512**	**1.643155**	**OK**
1024	1024	1.661475	OK

Сравнение gpu и cpu

Image: 256 x 256, classes: 8
CPU time: 6.067726 ms
GPU kernel time: 0.140589 ms
Speedup: 43.159385
Blocks: 256
Threads: 256
Results: OK

Image: 1024 x 1024, classes: 16
CPU time: 169.368020 ms
GPU kernel time: 9.170397 ms
Speedup: 18.468996

Blocks: 4096
Threads: 256
Results: OK

Выводы

Данная лабораторная работа познакомила нас с классификацией изображений на GPU с использованием CUDA и константной памяти. В ходе работы был реализован метод минимального расстояния: для каждого пикселя определялся класс с ближайшим средним цветом в пространстве RGB. Средние цвета классов вычислялись на CPU по обучающим пикселям и переносились в константную память GPU для совместного использования потоками. Основное внимание при реализации потребовалось уделить распределению пикселей между потоками, вычислению расстояний и сохранению номера класса в четвёртом байте пикселя. Работа позволила закрепить навыки использования константной памяти, передачи данных между CPU и GPU и запуска CUDA-ядер. Эффективность такого подхода зависит от размера изображения и количества классов: при небольшом объёме вычислений затраты на передачу данных и запуск ядра могут превышать выигрыш от параллельной обработки, а при увеличении нагрузки использование GPU позволяет сократить время классификации.